Lời thuyết minh video demo luận văn

Luận văn Thạc sĩ · Bùi Văn Dũng

Hai video nằm trong `output/videos/`. Nên chiếu **pha A trước, pha B sau**, đúng thứ tự hai pha của luận văn:

Video	File	Thời lượng	Trả lời
Pha A: huấn luyện phân tán	`LuanVan_demo_phaseA.mp4`	1:36	RQ1, RQ2: mô hình được chọn và huấn luyện trên cụm
Pha B: triển khai biên	`LuanVan_demo.mp4`	4:36	RQ3: hệ thống chạy thời gian thực trên hai Jetson

Mọi lệnh và mọi dòng kết quả trong video được ghi từ một lần chạy thật ngày 11/10/2026 (pha B lúc 21:36–21:44, pha A lúc 22:19–22:35). Khi biên tập video, chỉ có ba thay đổi:

- các đoạn chờ dài (Spark nạp mô hình, huấn luyện) được rút ngắn;
- Spark UI ở pha A được tua nhanh 31× (có ghi trên màn hình);
- hai dòng nhiễu bị ẩn: traceback `KeyboardInterrupt` của PySpark sau khi tắt (pha B), và dòng `Model Size` đo sai trong chế độ cụm (pha A, xem phần hỏi đáp).

Các cảnh ngắn (dưới 10 giây) có ít thời gian để nói. Nếu cần nói hết ý, **bấm tạm dừng** ở cuối cảnh rồi chạy tiếp.

Video 1 · Pha A: Huấn luyện mô hình phân tán trên cụm Spark (1:36)

0:00–0:06 · Tiêu đề

> Đầu tiên em xin trình bày pha A: huấn luyện mô hình trên cụm Spark Standalone. Máy chủ đóng vai trò Spark Master, còn hai Jetson Orin Nano Super là hai Worker.

0:06–0:11 · Bước 1: Khởi động Spark Master

Trên màn hình: lệnh `./cluster/start_master_mac.sh`. Master lắng nghe ở `spark://192.168.1.165:7077`, giao diện web ở cổng 8080.

> Trên máy chủ, em khởi động Spark Master. Đây là nút điều phối, nhận các worker và phân phối công việc cho chúng.

0:11–0:26 · Bước 2: Khởi động Spark Worker trên hai Jetson

Trên màn hình: hai ô Jetson #1 (192.168.1.81) và Jetson #2 (192.168.1.39), mỗi ô chạy `ssh`, rồi `./cluster/start_worker.sh`.

Chỉ vào: dòng `Starting Spark worker … with 4 cores, 5.0 GiB RAM`, dòng `Running Spark version 3.4.1` và dòng cuối `Successfully registered with master`.

> Trên mỗi Jetson, em khởi động một Spark Worker. Mỗi worker nhận 4 nhân CPU và 5 GB bộ nhớ, rồi đăng ký với Master. Dòng cuối cho thấy cả hai worker đã đăng ký thành công.

0:26–0:37 · Bước 3: Kiểm tra cụm

Trên màn hình: `python demo/cluster_status.py` báo `alive workers: 2`, tổng `8 cores, 10 GiB`. Sau đó là Spark Master UI với hai worker ở trạng thái ALIVE.

> Kiểm tra lại, cụm có hai worker đang hoạt động, tổng cộng 8 nhân và 10 GB bộ nhớ. Giao diện Spark Master cũng hiện hai worker Jetson sẵn sàng nhận việc.

0:37–0:57 · Bước 4: Gửi job huấn luyện lên cụm

Trên màn hình: lệnh `./cluster/run_ml_remote.sh demo/train_demo.py`, kèm các biến `CLUSTER_DRIVER=bvdung@192.168.1.39`, bộ nhớ driver và executor 2 GB.

Chỉ vào (theo thứ tự):

1. Khối `SPARK CLUSTER`: có **hai executor**, một trên 192.168.1.39 và một trên 192.168.1.81, mỗi executor 4 nhân.
2. Khối `LOADING DATA FROM PARQUET`: **1.785.349** mẫu huấn luyện, **447.275** mẫu kiểm tra.
3. Dòng `Port features excluded to avoid label leakage`: còn **77** đặc trưng không rò rỉ.
4. Khối `FEATURE SELECTION: SHAP TOP-30`: 77 đặc trưng được rút còn **30**.
5. Dòng `TRAINING: Random Forest (200 trees, max depth 15)`.

> Em gửi job huấn luyện lên cụm. Driver đặt trên Jetson số 2, còn executor chạy trên cả hai Jetson. Dữ liệu là tập CICIDS2017 đã qua quy trình loại rò rỉ: bỏ các đặc trưng cổng, khử trùng lặp, còn khoảng 1,79 triệu mẫu huấn luyện và 447 nghìn mẫu kiểm tra với 77 đặc trưng. Từ 77 đặc trưng, em giữ 30 đặc trưng quan trọng nhất theo SHAP, tức giảm khoảng 60%. Đây chính là cấu hình trả lời câu hỏi nghiên cứu số 1. Mô hình là Random Forest 200 cây, độ sâu tối đa 15, đúng mô hình được triển khai xuống thiết bị biên ở pha B.

0:57–1:27 · Spark UI: job chạy trên hai executor (tua nhanh 31×)

Trên màn hình: Spark UI của ứng dụng (cổng 4040 trên driver), danh sách job `collectAsMap at RandomForest.scala`, mỗi job 16/16 task.

> Đây là giao diện Spark của job, tua nhanh 31 lần. Mỗi dòng là một vòng Spark tìm điểm chia tốt nhất cho một nhóm nút của các cây. Mỗi vòng chia thành 16 task, chạy song song trên executor của cả hai Jetson. Thời gian thật của bước huấn luyện là khoảng 14 phút.

1:27–1:36 · Kết quả

Chỉ vào: khối `RESULTS: Random Forest – SHAP Top-30 (full test split)`.

Chỉ số	Giá trị
Accuracy	0,998710
Precision	0,995063
Recall	0,996352
F1-Score	**0,995707**
AUC-ROC / AUC-PR	0,999948 / 0,999775
TP / TN / FP / FN	66.911 / 379.787 / 332 / 245
Thời gian huấn luyện	856,9 s

> Trên toàn bộ tập kiểm tra, mô hình đạt F1 khoảng 0,9957 và AUC-ROC 0,9999, khớp với kết quả em báo cáo trong luận văn. Đây là mô hình em sẽ đưa xuống thiết bị biên trong pha B.

Video 2 · Pha B: Triển khai biên thời gian thực trên hai Jetson (4:36)

0:00–0:06 · Tiêu đề

> Tiếp theo là pha B: triển khai hệ thống phát hiện xâm nhập thời gian thực trên cụm hai Jetson, dùng Kafka và PySpark.

0:06–0:14 · Bước 1: Hạ tầng trên máy chủ

Trên màn hình: `docker compose ps` liệt kê 5 container: `ids-kafka`, `ids-zookeeper`, `ids-postgres`, `ids-influxdb`, `ids-grafana`. Lệnh `kafka-topics --list` trả về hai topic `ids-network-flow` và `ids-suspicious-flow`.

> Trên máy chủ có Kafka để truyền luồng dữ liệu, PostgreSQL để lưu kết quả dự đoán, InfluxDB để lưu số liệu hiệu năng và Grafana để giám sát. Có hai topic: `ids-network-flow` chứa toàn bộ luồng mạng đi vào, `ids-suspicious-flow` chỉ chứa các luồng mà cổng lọc đánh giá là đáng ngờ.

0:14–0:23 · Bước 2: Kiểm tra hai Jetson

Chỉ vào: IP của từng board, dòng `NV Power Mode: 25W`, các file mô hình (`anomaly_autoencoder.pkl` trên Jetson #1, `ids_pipeline_model` trên Jetson #2).

> Hai Jetson chạy ở chế độ nguồn 25 W. Jetson số 1 chứa mô hình autoencoder của cổng lọc. Jetson số 2 chứa mô hình PySpark vừa huấn luyện ở pha A.

0:23–0:36 · Bước 3: Khởi động cổng lọc trên Jetson #1

Chỉ vào: `[OK] AnomalyScorer loaded`, `Features: 77`, `Threshold: 0.202087`, rồi `ANOMALY GATE READY`.

> Trên Jetson số 1, em khởi động cổng lọc bất thường. Đây là một autoencoder dùng 77 đặc trưng, huấn luyện chỉ trên lưu lượng lành tính. Luồng nào có sai số tái tạo vượt ngưỡng thì bị coi là đáng ngờ và được chuyển sang bộ phân loại. Các luồng còn lại được kết luận lành tính ngay tại cổng lọc.

0:36–0:58 · Bước 4: Khởi động bộ phân loại PySpark trên Jetson #2

Chỉ vào: `Spark Session created (version: 3.4.1, master: local[*])`, `Feature columns loaded: 30 features`, `Pipeline stages: VectorAssembler, StandardScalerModel, RandomForestClassificationModel`, rồi `CLASSIFIER READY`.

> Trên Jetson số 2, em khởi động bộ phân loại. Nó nạp PipelineModel của Spark gồm ba bước: gom đặc trưng, chuẩn hóa và Random Forest, đúng mô hình 30 đặc trưng SHAP vừa huấn luyện. Bộ phân loại chỉ đọc topic luồng đáng ngờ.

0:58–1:44 · Bước 5: Phát lại CICIDS2017 và log trực tiếp

Trên màn hình: ba ô. Ô trên là máy chủ chạy `python sender/data_sender.py … --rate 15`. Hai ô dưới là log của hai Jetson.

Chỉ vào: dòng `[MONITOR:jetson-nano-1] … Throughput: ~13–15 rps | Latency: ~0.25ms`, và dòng `[MONITOR:jetson-nano-2] … Attacks: …` với số tấn công tăng dần.

> Từ máy chủ, em phát lại các luồng mạng của tập kiểm tra CICIDS2017 vào Kafka với tốc độ 15 luồng mỗi giây. Cổng lọc trên Jetson số 1 xử lý khoảng 13 đến 15 luồng mỗi giây, mỗi luồng mất khoảng 0,25 ms. Bộ phân loại trên Jetson số 2 chỉ nhận phần nhỏ đáng ngờ và bắt đầu báo các luồng tấn công.

1:44–4:14 · Kết quả trên Grafana

Trên màn hình: dashboard Grafana. Cảnh này dài khoảng 2,5 phút nên có thể nói thong thả.

Chỉ vào (theo thứ tự):

1. **Prediction Throughput:** thông lượng của từng nút. Cổng lọc khoảng 15 req/s, bộ phân loại thấp hơn nhiều vì chỉ nhận luồng đáng ngờ.
2. **Attack Detection Count:** số tấn công phát hiện được theo thời gian.
3. **Avg Inference Latency:** độ trễ suy luận trung bình trên mỗi luồng. Cổng lọc chỉ mất khoảng 0,25 ms, còn bộ phân loại PySpark mất vài trăm ms, vì mỗi lô vài chục luồng tốn vài giây.
4. **CPU / Memory:** tải của từng Jetson. Jetson số 2 (Spark) có các đỉnh CPU cao hơn hẳn Jetson số 1.
5. **Recent Alerts:** các cảnh báo `ATTACK_DETECTED` mới nhất, được ghi vào PostgreSQL.

> Đây là kết quả cuối cùng mà người vận hành nhìn thấy: dashboard Grafana cập nhật theo thời gian thực. Thông lượng cho thấy cổng lọc gánh toàn bộ lưu lượng, còn bộ phân loại Spark chỉ xử lý phần đáng ngờ. Đó là lý do tách hai tầng ra hai bo mạch giúp hệ thống chịu tải cao hơn. Biểu đồ tấn công và bảng cảnh báo cho thấy các luồng tấn công được phát hiện và lưu lại gần như ngay khi xuất hiện. Biểu đồ CPU cho thấy tải tập trung ở Jetson số 2, nơi chạy Spark. Đây là nút thắt mà luận văn đã chỉ ra, và là lý do hướng phát triển chuyển phần suy luận sang ONNX Runtime.

4:14–4:22 · Bước 6: Dừng hệ thống và thống kê cuối

Chỉ vào:

- Máy chủ: `Total sent: 2,097 rows`, `Avg rate: 14.1 rows/s`.

- Jetson #1: `Total scored: 2,097`, `Total forwarded: 235`.
- Jetson #2: `Total predictions: 235`, `Total attacks: 226`.

> Em dừng hệ thống bằng Ctrl+C. Trong khoảng 2,5 phút, máy chủ đã gửi 2.097 luồng. Cổng lọc chấm điểm toàn bộ 2.097 luồng nhưng chỉ chuyển 235 luồng, khoảng 11%, sang bộ phân loại. Bộ phân loại kết luận 226 trong số đó là tấn công.

4:22–4:36 · Bước 7: Kiểm chứng với nhãn thật

Chỉ vào: bảng SQL đếm dự đoán theo đường đi, rồi khối `VERIFY RUN AGAINST CICIDS2017 LABELS`.

Kết quả	Giá trị
Luồng đã gửi / dự đoán đã lưu	2.097 / 2.097
Cổng lọc bỏ qua	1.862 (88,8%)
Tấn công thật đã gửi	328
Phát hiện đúng	**226, recall 68,9%**
Bỏ sót do cổng lọc / do PySpark	102 / 0
Báo động giả trên 1.769 luồng lành tính	**0**
DoS Hulk / DDoS	147/174 / 75/123

> Cuối cùng, em đối chiếu từng dự đoán đã lưu với nhãn thật của CICIDS2017. Cổng lọc bỏ qua 88,8% lưu lượng. Hệ thống phát hiện đúng 226 trên 328 luồng tấn công, recall 68,9%, và không có báo động giả nào trên gần 1.800 luồng lành tính. Toàn bộ tấn công bị bỏ sót đều nằm ở cổng lọc. Bộ phân loại Spark không bỏ sót tấn công nào trong số luồng nó nhận được. Đây chính là đánh đổi em đã nêu trong luận văn: cổng lọc giúp tăng khoảng 9 lần năng lực xử lý nhưng phải trả bằng khoảng 30% recall. Ngưỡng của cổng lọc có thể điều chỉnh theo đường cong vận hành để cân bằng lại hai yếu tố này.

Câu hỏi hội đồng có thể đặt khi xem demo

Vì sao recall chỉ khoảng 69% trong khi F1 của mô hình là 0,9957? F1 0,9957 là của bộ phân loại Random Forest trên toàn bộ tập kiểm tra. Trong hệ thống biên, luồng phải qua cổng lọc trước. Cổng lọc bỏ qua khoảng 89% lưu lượng để giảm tải cho Spark, và một phần tấn công (102/328 trong demo) bị bỏ qua cùng. Phần đến được bộ phân loại thì không bị bỏ sót (0 trường hợp). Ngưỡng cổng lọc là một điểm trên đường cong vận hành: hạ ngưỡng sẽ tăng recall nhưng chuyển nhiều luồng hơn sang Spark.

Vì sao bộ phân loại chậm (trung bình vài trăm ms mỗi luồng, 580,7 ms ở thống kê cuối)? Mỗi lần gọi PySpark tốn chi phí cố định khoảng 7 giây trên Jetson. Chia cho vài chục luồng trong một lô thì ra vài trăm ms mỗi luồng. Luận văn đo bằng cách quét kích thước lô: thời gian mỗi lô gần như không đổi từ 10 đến 1.000 luồng. Chạy cùng mô hình bằng ONNX Runtime nhanh hơn khoảng 7.200 lần ở lô 10, nên đó là hướng phát triển tiếp theo.

Vì sao driver đặt trên Jetson #2 ở pha A? Lúc quay, Jetson #1 đang chạy tác vụ khác và chỉ còn khoảng 2,6 GB RAM trống. Cấu hình mặc định của cụm đặt driver ở Jetson #1; script cho phép đổi qua biến `CLUSTER_DRIVER`. Kết quả huấn luyện không phụ thuộc vào việc driver đặt ở đâu.

Vì sao trong video không có kích thước mô hình? Trong chế độ cụm, `model.save()` ghi dữ liệu cây ra ổ đĩa riêng của từng executor (cụm không có ổ đĩa chung), nên hàm đo trên driver chỉ thấy phần metadata (0,028 MB). Kích thước thật của mô hình khoảng 4,4 MB, như trong luận văn. Dòng đo sai này được ẩn khỏi video.

Video có bị dựng hay chỉnh số liệu không? Không. Mọi lệnh và dòng kết quả đều được ghi từ lần chạy thật, kèm mốc thời gian. Chỉ có các đoạn chờ được rút ngắn, Spark UI được tua nhanh (có ghi trên màn hình), và hai dòng nhiễu nêu ở đầu tài liệu bị ẩn. Script quay nằm trong `demo/` (`run_session.py`, `run_session_a.py`); có thể chạy lại để ra kết quả tương tự.

Tốc độ 15 luồng/giây có phải giới hạn của hệ thống? Không. Demo chọn 15 luồng/giây để chạy ổn định dưới thông lượng tối đa. Luận văn đo được chế độ tách hai tầng chịu được khoảng 22 luồng/giây, gấp khoảng 9 lần một nút chạy toàn bộ pipeline.